



MACHINE LEARNING IN BIOINFORMATICS

Part 2: Unsupervised learning

František Mráz
KSVI MFF UK



INTRODUCTION

March 10, 2026

Machine Learning in Bioinformatics

3

fppt.com

Part 2: Unsupervised learning PROBABILITY DENSITY ESTIMATION APPROACHES

March 10, 2026

Machine Learning in Bioinformatics

10

fppt.com

Part 2: Unsupervised learning DIMENSIONALITY REDUCTION

March 10, 2026

Machine Learning in Bioinformatics

27

fppt.com



INTRODUCTION

Sources



- Yang, Zheng Rong. *Machine learning approaches to bioinformatics*. Science, Engineering, and Biology Informatics — Vol. 4. World scientific, 2010.
 - Accessible via library [EBSCO e-books](#) – **search by title**
- Bishop, Christopher M. *Pattern recognition and machine learning*. Vol. 4. No. 4. New York: Springer, 2006.
 - [Available from the author](#) (look for the blue button “Free PDF”)

Learning for Analysis of Biological Data



- **Example:** Plants generate many thousands of metabolites (intermediates and products of metabolism; the term *metabolite* is usually restricted to small molecules ≤ 900 daltons (1 dalton = weight of a nucleon (neutron or proton))).
 - A mass spectrometry experiment can measure mass of each metabolite and number of its replicates.
 - Based on masses it is possible to infer a number of chemical formulas of candidate compounds from different pathways
 - However, one metabolite can be mapped to multiple compounds
- **Supervised learning**
 - Find a mapping from one data space to another data space
Chemical formulas $\rightarrow$ *pathways*
- **Unsupervised learning**
 - One data space is missing – reorganize the data space to explore the missing data space

SK: signálne dráhy

Why Unsupervised Learning



Example: Analysis of gene expression

- Microarray experiment can monitor simultaneously thousands of genes and proteins under different experimental conditions – for studying genome- and proteome-wide functions and regulatory mechanisms
- **Dimensionality reduction** – to reduce the dimensionality of the 'gene space' (e.g., in microarray data) by constructing 'super-genes' – to simplify structure of the data
- **Visualizations** – reduce the data to 2 or 3 dimensions (e.g., principal components analysis)
- **Clustering** – partitioning the data into groups of objects 'similar' to each other so that objects in different groups are 'dissimilar' – identifying biologically relevant groups of both genes and samples ⇒ insight into gene function and regulation

Subjects of Unsupervised Learning



1. Density estimation

- Find information hidden in the data
- E.g., the data can be generated from a normal distribution (i.e., x_i is a real value) and we can find the parameters of the Gaussian (mean and standard deviation)

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad \sigma = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu)^2}$$

Sample mean

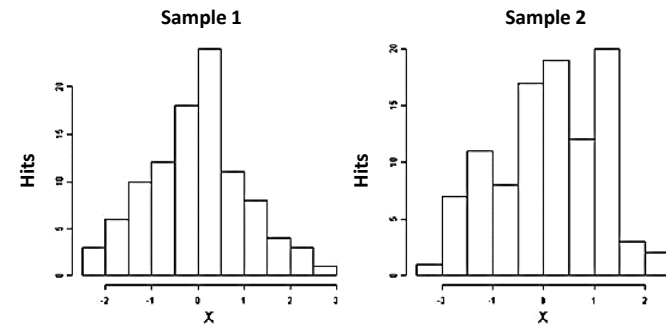
Sample standard deviation

where $x_1, x_2, \dots, x_N$ are the data points, $N \geq 1$

- Gaussian distribution is regarded as data structure and parameters are regarded as inference rules
- Usually, methods from statistics

2. Dimensionality reduction

3. Cluster analysis



Both samples are from the same probability distribution $\mathcal{N}(0,1)$, but mean and standard deviation are -0.015 and 1.052 for Sample 1, and 0.074 and 1.009 for Sample 2.

Subjects of Unsupervised Learning

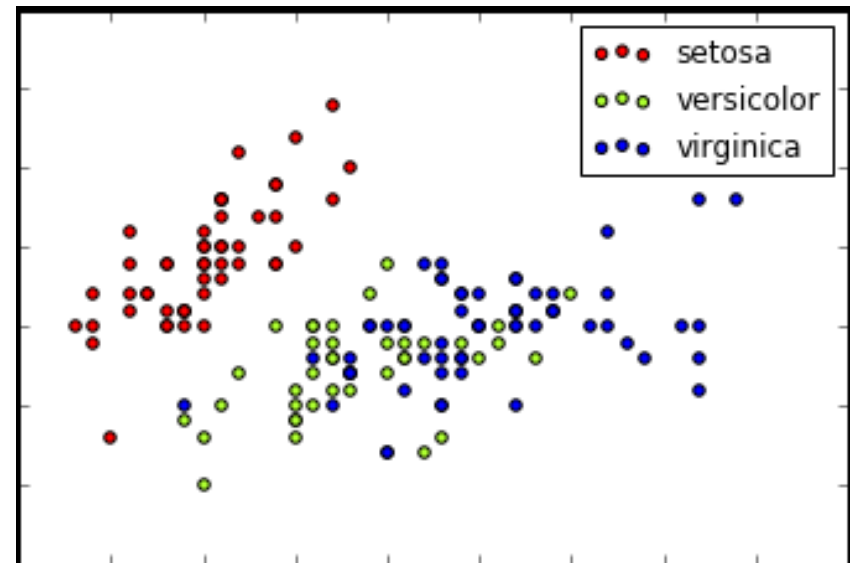


1. Density estimation

2. Dimensionality reduction

- Often many-dimensional data is not possible to visualize
- E.g., gene expression data for a disease may contain only a few samples, but with $1000 \sim 100000$ genes as variables

3. Cluster analysis



Mapping 4-dimensional Iris data to a 2-dimensional space. Setosa is well separated from the other two species, which are difficult to separate.

Subjects of Unsupervised Learning



1. Density estimation

2. Dimensionality reduction

3. Cluster analysis

- A data set may be viewed as a composition of disjointed sub-structures.
- Each sub-structure contains data points with similar properties.
- How to find these sub-structures and quantitatively describe them?



Part 2: Unsupervised learning

PROBABILITY DENSITY ESTIMATION APPROACHES

Probability Density Estimation Approaches



- Types of estimations
 1. The histogram approach – the simplest method
 2. A parametric approach – assumes a structure in data (e.g., normal distribution); training data not kept
 3. Non-parametric approaches – no explicit data structure
 4. Semi-parametric approach

1. Histogram Approach



- Each coordinate is divided into segments of a fixed length (called bins)
- If $x \in [a, b]$, the interval is divided into K bins of length

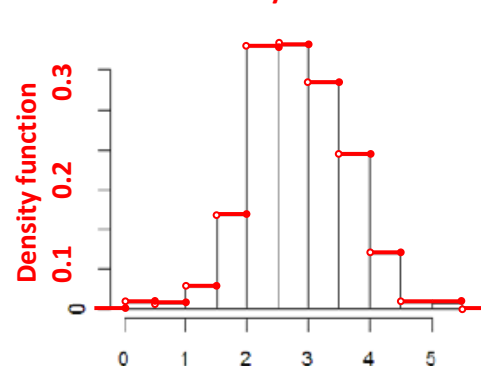
$$\Delta = \frac{b - a}{K}$$

- Each training data point is assigned to the bin it belongs
- Frequency of each bin is

$$\frac{\text{\#training data falling into the bin}}{\text{\#all training data}}$$

- The frequency is used as the probability how likely a point falls into the bin – simple visualization is a histogram

Density function



n_i = #data points in bin i
 p_i = probability a point falls in bin i

$$p_i = \frac{n_i}{N} \quad ?$$

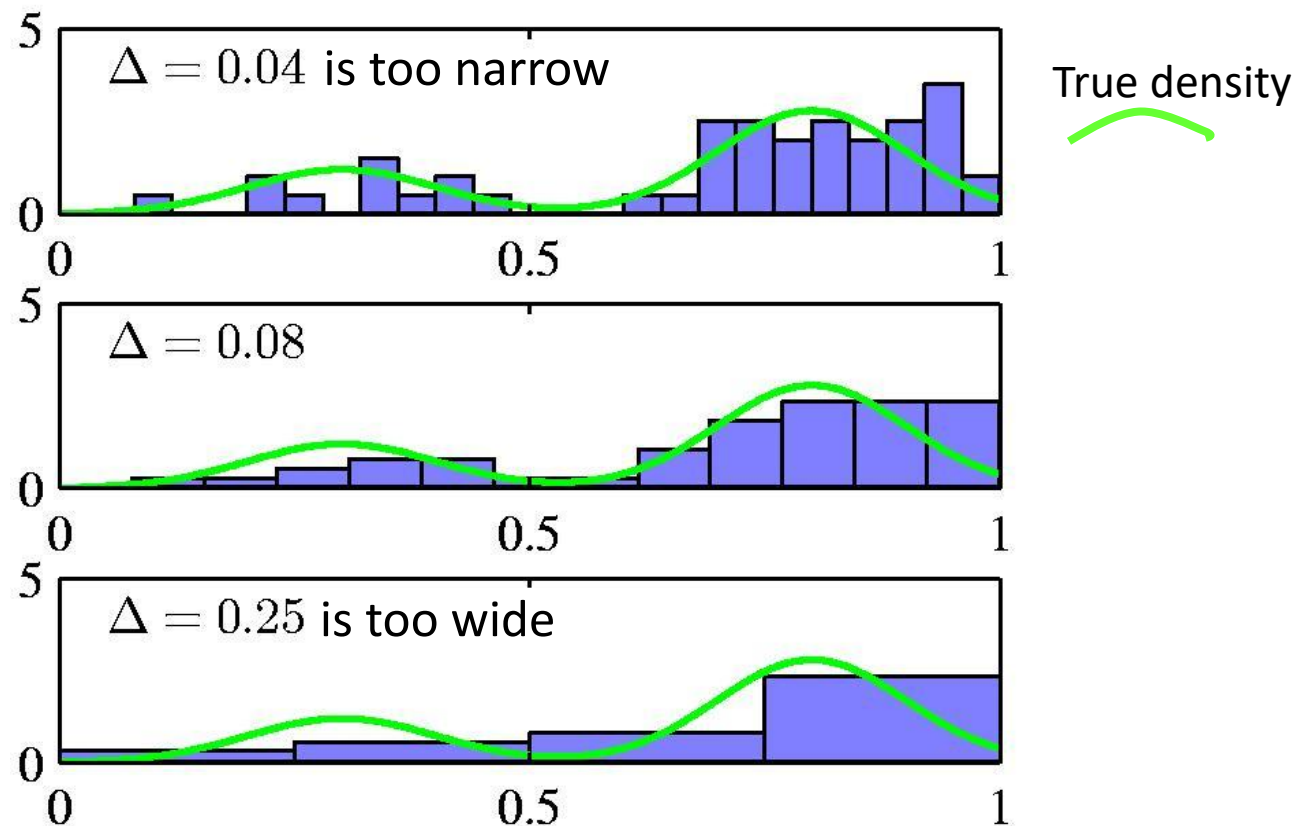
How to approximate the probability density function?

$$f_i = \frac{n_i}{N\Delta}$$

1. Histogram Approach



- The value of Δ is important



1. Histogram Approach

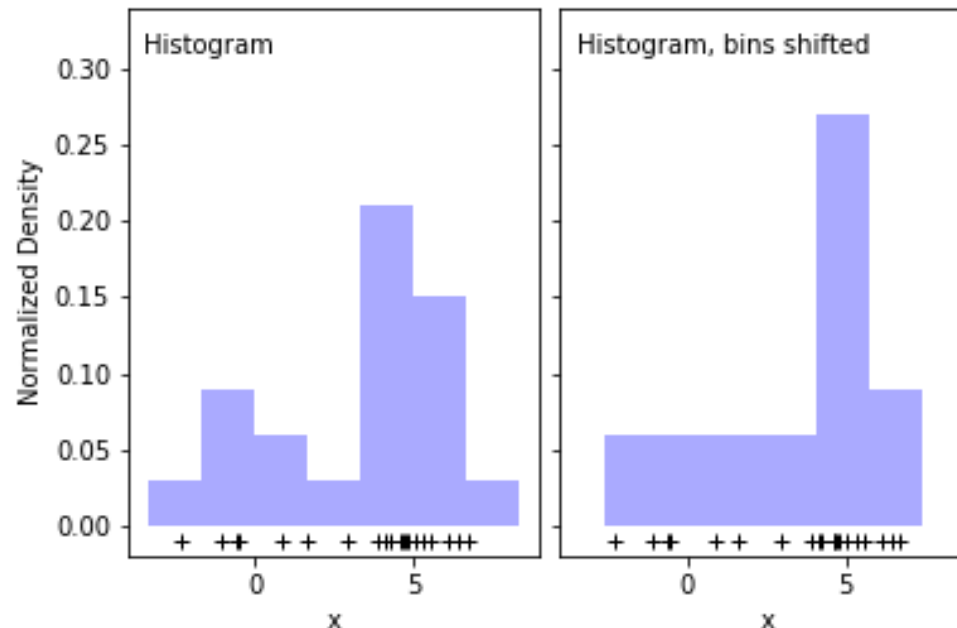


- Pros:
 - Simple method, no explicit data structure is supposed
 - No need to fit a model to the data
 - We just compute some very simple statistics (the number of data points in each bin) and store them
- Cons:
 - How wide should the bins be? (width=regularizer)
 - Do we want the same bin-width everywhere?
 - Do we believe the density is zero for empty bins?
 - for d variables we need K^d bins; e.g., for $K = 10$ and $d = 10$ we need 10^{10} bins
 - The density has discontinuities at the bin boundaries
 - We must be able to do better by some kind of smoothing

1. Histogram Approach



- Cons (cont.):
 - Positions of bin boundaries can change the outcome; see the example from the documentation for [scikit-learn](#)
 - On the left: bin boundaries `linospace(-5,10,10)`
 - On the right: bin boundaries `linospace(-4.25,10.75,10)`



2. Parametric Approach



- **Assumes** the data has a **structure** before estimating the probability
- After constructing the density probability function, the training data can be discarded
- If we assume a Gaussian data structure
 - A training set of size N is a sequence $\{x_i\}_{i=1}^N$, represented as a matrix $\mathbf{X}$,
 - $x_i = (x_{i1}, \dots, x_{id})^T \in \mathbb{R}^d$, (i.e., x_i is a column vector corresponding to i -th row of $\mathbf{X}$)
 - The training data points are generated by the assumed Gaussian data structure with likelihood $\mathcal{L}$

Likelihood that all N observations in X are from the Gaussian distribution $\mathcal{N}(\mu, \Sigma)$

1D Gaussian probability density function $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

$$\mathcal{L} = \prod_{i=1}^N p(x_i) \quad \text{where} \quad p(x_i) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} e^{-\frac{(x_i - \mu)^T \Sigma^{-1} (x_i - \mu)}{2}}$$

where $\Sigma \in \mathbb{R}^{d \times d}$ is the covariance matrix (see below), Σ^{-1} is its inverse and $|\Sigma|$ its determinant, $\mu \in \mathbb{R}^d$ is the mean vector

- We **maximize** the likelihood function

2. Parametric Approach – Multidimensional Gaussian



x, μ are a column vectors

- The **covariance** matrix $\Sigma = E[(x - \mu)(x - \mu)^T]$ for $x \in \mathbf{X}$, i.e.
- $\Sigma_{ij} = cov(\mathbf{x}_i, \mathbf{x}_j) = E[(\mathbf{x}_i - \mu_i)^T(\mathbf{x}_j - \mu_j)]$
- $\mu = \frac{1}{N} \sum_{k=1}^N x_k \quad \mu_i = \frac{1}{N} \sum_{k=1}^N x_{ki}$

$$\mathbf{x}_j = \begin{pmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{Nj} \end{pmatrix} \quad \mathbf{X} = (\mathbf{x}_1 \mathbf{x}_2 \cdots \mathbf{x}_d) = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix}$$
$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & & x_{2d} \\ & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{Nd} \end{pmatrix}$$

1D sample variance for a vector $y = (y_1, \dots, y_n)^T$ is $\frac{1}{N-1} \sum_{i=1}^n (y_i - \mu_y)^2 = \frac{1}{N-1} (y - \mu_y)^T (y - \mu_y)$

- Write formula for computing Σ_{ij} in terms of x_{rs} !

2. Parametric Approach – Multidimensional Gaussian



x, μ are a column vectors

- The **covariance** matrix $\Sigma = E[(x - \mu)(x - \mu)^T]$ for $x \in \mathbf{X}$, i.e.
$$\Sigma_{ij} = cov(\mathbf{x}_i, \mathbf{x}_j) = E[(\mathbf{x}_i - \mu_i)^T(\mathbf{x}_j - \mu_j)]$$
- What is the correct formula for the **sample variance matrix**, also called the **covariance matrix** or the variance-covariance matrix?

$a) \quad \Sigma_{ij} = \frac{1}{d} \sum_{k=1}^d (x_{ki} - \mu_i)(x_{kj} - \mu_j)$	$f) \quad \Sigma_{ij} = \frac{1}{d-1} \sum_{k=1}^d (x_{ki} - \mu_i)(x_{kj} - \mu_j)$
$b) \quad \Sigma_{ij} = \frac{1}{N} \sum_{k=1}^N (x_{ki} x_{kj} - \mu_i \mu_j)$	$g) \quad \Sigma_{ij} = \frac{1}{N-1} \sum_{k=1}^N (x_{ki} x_{kj} - \mu_i \mu_j)$
$c) \quad \Sigma_{ij} = \frac{1}{N} (\mathbf{x}_i - \mu_i)^T (\mathbf{x}_j - \mu_j)$	$h) \quad \Sigma_{ij} = \frac{1}{N-1} (\mathbf{x}_i - \mu_i)^T (\mathbf{x}_j - \mu_j)$
$d) \quad \Sigma_{ij} = \mathbf{x}_i^T \mathbf{x}_j - \mu_i \mu_j$	$i) \quad \Sigma_{ij} = \frac{1}{N-1} \mathbf{x}_i^T \mathbf{x}_j - \mu_i \mu_j$
$e) \quad \Sigma_{ij} = \frac{1}{d} (\mathbf{x}_i - \mu_i)^T (\mathbf{x}_j - \mu_j)$	$j) \quad \Sigma_{ij} = \frac{1}{N-1} \sum_{k=1}^N (x_{ki} - \mu_i)(x_{kj} - \mu_j)$



2. Parametric Approach – Multidimensional Gaussian



x, μ are a column vectors

- The **covariance** matrix $\Sigma = E[(x - \mu)(x - \mu)^T]$ for $x \in \mathbf{X}$, i.e.
$$\Sigma_{ij} = \text{cov}(\mathbf{x}_i, \mathbf{x}_j) = E[(\mathbf{x}_i - \mu_i)^T(\mathbf{x}_j - \mu_j)]$$
- What is the correct formula for the **sample variance matrix**, also called the **covariance matrix** or the variance-covariance matrix?

a) $\Sigma_{ij} = \frac{1}{d} \sum_{k=1}^d (x_{ki} - \mu_i)(x_{kj} - \mu_j)$	f) $\Sigma_{ij} = \frac{1}{d-1} \sum_{k=1}^d (x_{ki} - \mu_i)(x_{kj} - \mu_j)$
b) $\Sigma_{ij} = \frac{1}{N} \sum_{k=1}^N (x_{ki} x_{kj} - \mu_i \mu_j)$	g) $\Sigma_{ij} = \frac{1}{N-1} \sum_{k=1}^N (x_{ki} x_{kj} - \mu_i \mu_j)$
c) $\Sigma_{ij} = \frac{1}{N} (\mathbf{x}_i - \mu_i)^T (\mathbf{x}_j - \mu_j)$	h) $\Sigma_{ij} = \frac{1}{N-1} (\mathbf{x}_i - \mu_i)^T (\mathbf{x}_j - \mu_j)$
d) $\Sigma_{ij} = \mathbf{x}_i^T \mathbf{x}_j - \mu_i \mu_j$	i) $\Sigma_{ij} = \frac{1}{N-1} \mathbf{x}_i^T \mathbf{x}_j - \mu_i \mu_j$
e) $\Sigma_{ij} = \frac{1}{d} (\mathbf{x}_i - \mu_i)^T (\mathbf{x}_j - \mu_j)$	j) $\Sigma_{ij} = \frac{1}{N-1} \sum_{k=1}^N (x_{ki} - \mu_i)(x_{kj} - \mu_j)$



2. Parametric Approach – Multidimensional Gaussian



x, μ are a column vectors

- The **covariance** matrix $\Sigma = E[(x - \mu)(x - \mu)^T]$ for $x \in \mathbf{X}$, i.e.
$$\Sigma_{ij} = cov(\mathbf{x}_i, \mathbf{x}_j) = E[(\mathbf{x}_i - \mu_i)^T(\mathbf{x}_j - \mu_j)]$$
- If the training data are orthogonal (i.e., all attributes are independent), the covariance matrix becomes diagonal

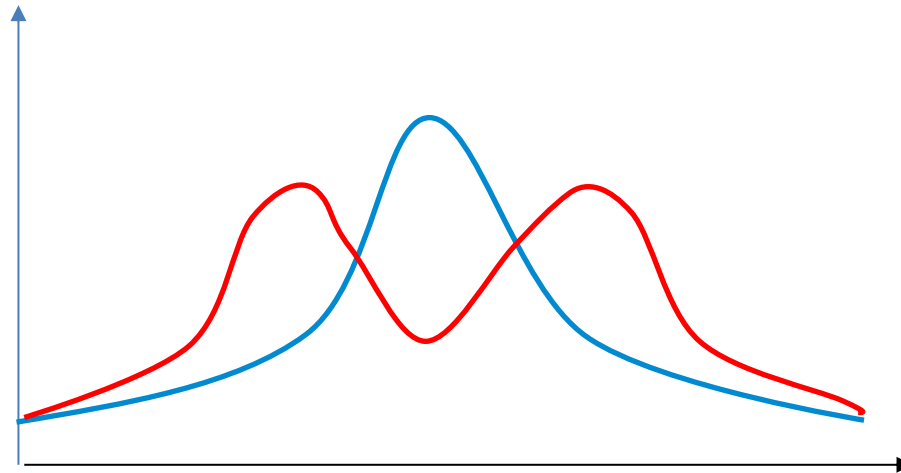
$$\Sigma = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_d^2 \end{pmatrix}$$

- If the data are orthogonal and homogenous (i.e., $\sigma_1 = \sigma_2 = \cdots = \sigma_d = \sigma$)
 - $\Sigma = \sigma^2 \mathbf{I}$ where $\mathbf{I}$ is the identity matrix
 - $\mu = \frac{1}{N} \sum_{i=1}^N x_i$ and $\sigma^2 = \frac{1}{(N-1)d} \sum_{i=1}^N (x_i - \mu)^T (x_i - \mu)$

2. Parametric Approach



- Parametric distribution models are restricted to specific forms, which may not always be suitable; for example, consider modelling a **multimodal distribution** with a single, **unimodal model**.



3. Non-Parametric Approach



- Building a model without clearly defined structure
- For a prediction all (training) data are needed
- **Local density estimators** $\approx$ estimate the density in a small region

$$p(x) = \frac{K}{NV}$$

where K is the number of points in the region, V is the volume of the region, N is the total number of points

- Two common methods
 - A. Kernel approach
 - B. K -nearest neighbor approach

3A. Kernel Density Estimators



- Use regions centered on the data points
 - Allow regions to overlap
 - Each region contributes by $1/N$ to the total density
 - Avoiding discontinuities by using regions with soft edges

$$p(x) = \frac{1}{N} \sum_{i=1}^N \rho_i(x)$$

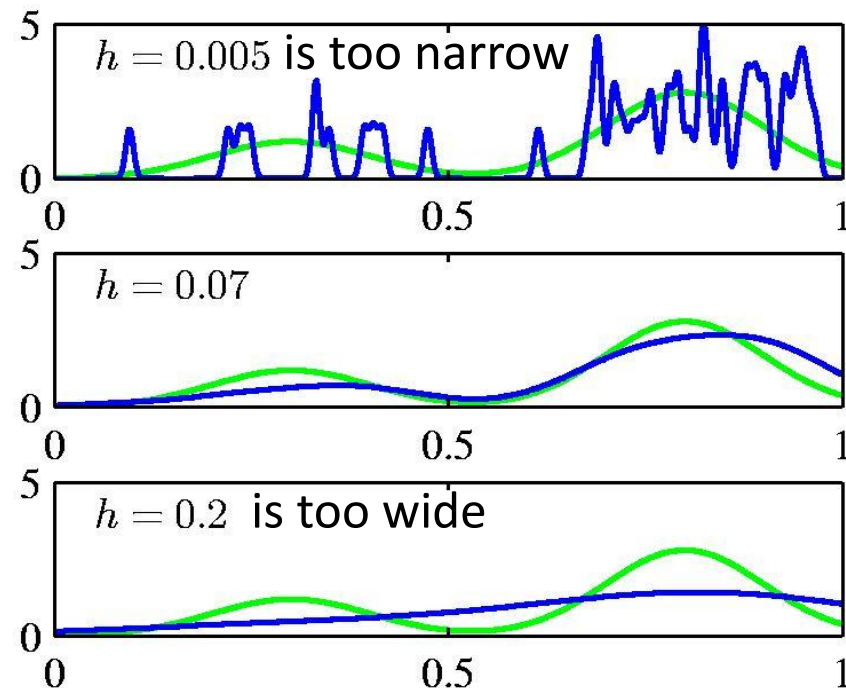
Where $\rho_i(x)$ is the similarity between the testing data point x and the training data point x_i . E.g., radial basis function (or Gaussian)

$$\rho_i(x) = \frac{1}{(2\pi h^2)^{d/2}} \exp\left(-\frac{\|x - x_i\|^2}{2h^2}\right)$$

Kernel function

compare to 1D Gaussian
probability density function

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



3B. K -Nearest Neighbor Approach for Density Estimation

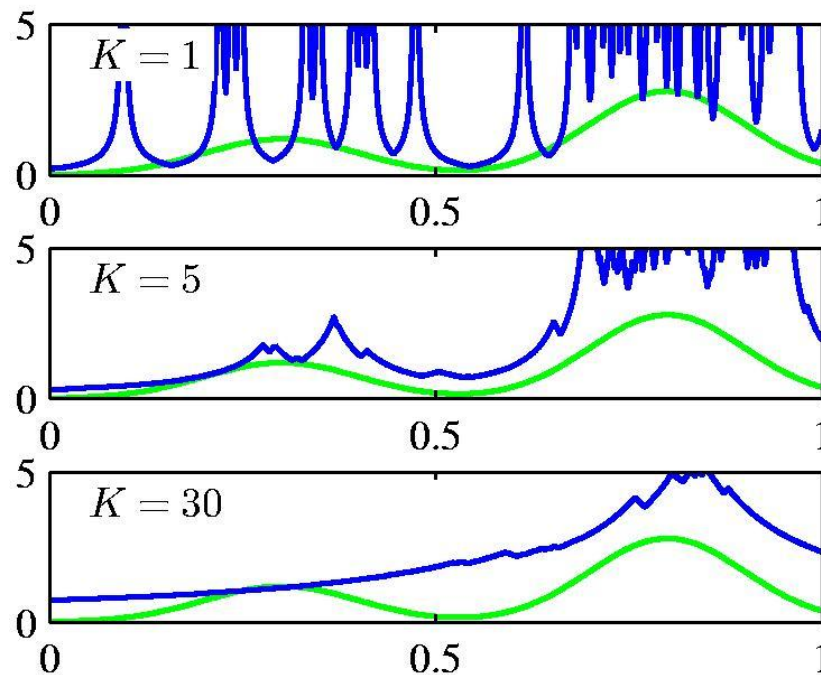


- Similarly, to the histogram approach
- Vary the size of a hyper-sphere around each test point so that exactly K training data points fall inside the hyper-sphere.
 - fix K , estimate V from the data. Consider a hyper-sphere centered on x and let it grow to a volume V^* , that includes K of the given N data points

$$p(x) = \frac{K}{NV^*}$$

- Does this give a fair estimate of the density?
- Nearest neighbors is usually used for classification or regression:
 - For regression, average the predictions of the K -nearest neighbors.
 - For classification, pick the class with the most votes.

3B. K -Nearest Neighbor Approach for Density Estimation



K acts as a smoother.

4. Semi-Parametric Approach



- We assume that data are generated from a model with M Gaussians
- Estimated density

$$f(x) = \sum_{m=1}^M w_m G_m(x)$$

where w_m , $0 \leq w_m \leq 1$ is the contribution of the m -th component

$$\sum_{m=1}^M w_m = 1$$

- $G_m(x)$ is the m -th component Gaussian

$$G_m(x) = \frac{1}{\sqrt{(2\pi)^d |\Sigma_m|}} e^{-\frac{(x_i - \mu)^T \Sigma_m^{-1} (x_i - \mu)}{2}}$$

- For fitting of such mixture model, the Expectation-Maximization (EM) algorithm is used [EM-algorithm is not presented here]

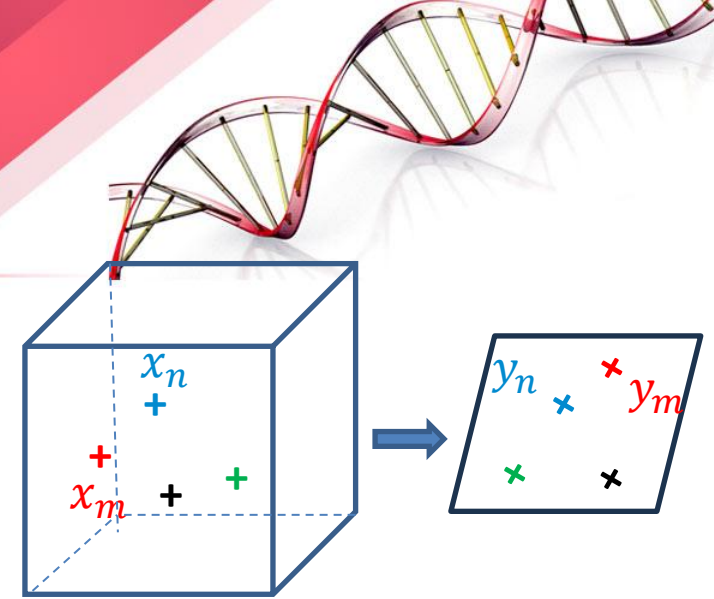


Part 2: Unsupervised learning

DIMENSIONALITY REDUCTION

Why

- Reduces time complexity: Less computation
- Reduces space complexity: Fewer parameters
- Saves the cost of observing the feature
- Simpler models are more robust on small datasets
- Better interpretable; simpler explanation
- Data visualization (structure, groups, outliers, etc.) if plotted in 2 or 3 dimensions

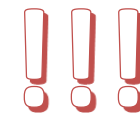


- A (one-to-one) mapping ϕ of the set of data points $\mathcal{D} = \{x_n \in \mathbb{R}^d\}_{n=1}^N$ where N is the number of data points, $d \geq 2$ is their dimension, to $\tilde{\mathcal{D}} = \{y_n \in \mathbb{R}^{\tilde{d}}\}_{n=1}^N$ and, where $\tilde{d} \leq 2$ is the new dimension;

$$\phi(x_n) = y_n, \quad \forall n \in \{1, \dots, N\}$$

- If x_m is the nearest neighbor of $x_n \in \mathcal{D}$, we expect that

$$\|y_m - y_n\| \leq \|y_i - y_n\|, \quad \forall i \in \{1, \dots, N\}$$



Feature Selection vs Extraction



- **Feature selection:**

- Choosing $k < d$ important features, ignoring the remaining $d - k$
 - *Subset selection algorithms*

- **Feature extraction (feature derivation):**

- Project the original $\mathbf{x}_i, i = 1, \dots, d$ dimensions to new $k < d$ dimensions, $\mathbf{z}_j, j = 1, \dots, k$
- Algorithms:
 - Matrix factorization
 - *Principal components analysis (PCA)*
 - *Linear discriminant analysis (LDA)*
 - *Factor analysis (FA)*
 - Graph layout
 - *Multidimensional scaling (MDS)*
 - *Isomap*
 - *Locally linear embedding*
 - *t-SNE*
 - *UMAP*

Subset Selection



- There are 2^d subsets of d features

1. Forward search:

- “Add the best feature at each step”
 - $F := \emptyset$
 - At each iteration, find the best new feature
$$j = \underset{i}{\operatorname{argmin}} \operatorname{Error}(F \cup \mathbf{x}_i)$$
 - Add $\mathbf{x}_j$ to F if $\operatorname{Error}(F \cup \mathbf{x}_j) < \operatorname{Error}(F)$
- Hill-climbing algorithm, trying $O(d^2)$ subsets of features

2. Backward search: Start with all features and remove one (which causes the least error) at a time

3. Floating search:

- Set the set of selected features $F := \emptyset$
- Set the set of possible features P to the set of all features
- Alternate adding k features to F and removing l features from P

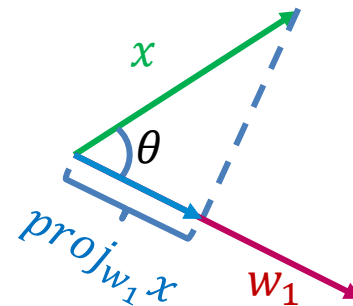
$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_d) = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{Nd} \end{pmatrix}$$

Principal Components Analysis (PCA)



- Find a low-dimensional space such that when x is projected there, information loss is minimized.
- The length of the projection of x on the direction (a column vector) w_1 is:

$$z = \frac{w_1^T x}{|w_1|}$$



$$\cos \theta = \frac{w_1 \cdot x}{|w_1| |x|} = \frac{w_1^T x}{|w_1| |x|}$$

the length of $\text{proj}_{w_1} x$ is $z = |x| \cdot \cos \theta = \frac{w_1^T x}{|w_1|}$

Principal Components Analysis (PCA)



- Find a low-dimensional space such that when x is projected there, information loss is minimized.
- The projection of x on the direction (a column vector) w_1 is:

$$z = w_1^T x$$

$$E[x] = \mu$$

$$\text{Var}(x) = E[(x - \mu)(x - \mu)^T] = \Sigma$$

is the covariance matrix

$$\text{Var}(z) = E[(z - E[z])^2]$$

is the variance of variable z

- Find w_1 such that $\text{Var}(z)$ is maximized

$$\begin{aligned}\text{Var}(z) &= \text{Var}(w_1^T x) = E[(w_1^T x - E[w_1^T x])^2] = E[(w_1^T x - w_1^T \mu)^2] = \\ &= E[(w_1^T x - w_1^T \mu)(w_1^T x - w_1^T \mu)] = E[w_1^T (x - \mu)(x - \mu)^T w_1] \\ &= w_1^T E[(x - \mu)(x - \mu)^T] w_1 = w_1^T \Sigma w_1\end{aligned}$$

This is a scalar, we can transpose it

Principal Components Analysis (PCA)



- Find a transformation matrix

$$\mathbf{W} = \left[\begin{pmatrix} w_1 \end{pmatrix} \begin{pmatrix} w_2 \end{pmatrix} \cdots \begin{pmatrix} w_k \end{pmatrix} \right]$$

we will use for the transformation

$$\mathbf{W}^T (x - \mu)$$

- Maximize variance of z

$$\text{Var}(z) = w_1^T \mathbf{\Sigma} w_1$$

- Without constraint we could pick a very big w_1 .

Principal Components Analysis (PCA)



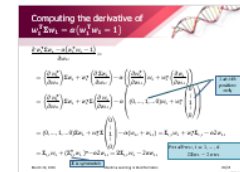
- Maximize $\text{Var}(z)$ subject to $\|w_1\| = 1$; we can use [Lagrange multiplier method](#) – introduce $\alpha > 0$ and

$$\max_{w_1} w_1^T \Sigma w_1 - \alpha (w_1^T w_1 - 1)$$

- At maximum, the derivative with respect to w_1 (all partial derivatives according to constituents of w_1) should be 0

$$2\Sigma w_1 - 2\alpha w_1 = 0$$

- Hence $\Sigma w_1 = \alpha w_1$ that is, w_1 is an eigenvector of Σ
- We want to maximize $\text{Var}(z) = w_1^T \Sigma w_1 = \alpha w_1^T w_1 = \alpha \Rightarrow$ choose the eigenvector with the largest eigenvalue λ_1
- Second principal component: $\max \text{Var}(z_2)$, s.t. $\|w_2\| = 1$ and w_2 is orthogonal to w_1



$$\max_{w_2} w_2^T \Sigma w_2 - \alpha (w_2^T w_2 - 1) - \beta (w_2^T w_1 - 0)$$

Computing the derivative of $w_1^T \Sigma w_1 - \alpha(w_1^T w_1 - 1)$



$$\frac{\partial w_1^T \Sigma w_1 - \alpha(w_1^T w_1 - 1)}{\partial w_{1i}} =$$

$$= \left(\frac{\partial w_1^T}{\partial w_{1i}} \right) \Sigma w_1 + w_1^T \left(\frac{\partial \Sigma w_1}{\partial w_{1i}} \right) - \alpha \left(\left(\frac{\partial w_1^T}{\partial w_{1i}} \right) w_1 + w_1^T \left(\frac{\partial w_1}{\partial w_{1i}} \right) \right)$$

1 at i -th position only

$$= \left(\frac{\partial w_1^T}{\partial w_{1i}} \right) \Sigma w_1 + w_1^T \Sigma \left(\frac{\partial w_1}{\partial w_{1i}} \right) - \alpha \left((0, \dots, 1, \dots, 0) w_1 + w_1^T \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix} \right)$$

$$= (0, \dots, 1, \dots, 0) \Sigma w_1 + w_1^T \Sigma \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix} - \alpha(w_{1i} + w_{1i}) = \Sigma_{i,*} w_1 + w_1^T \Sigma_{*,i} - \alpha 2w_{1i}$$

$$= \Sigma_{i,*} w_1 + (\Sigma_{*,i}^T w_1)^T - \alpha 2w_{1i} = 2\Sigma_{i,*} w_1 - 2\alpha w_{1i}$$

For all $w_{1i}, i = 1, \dots, d$
 $2\Sigma w_1 - 2\alpha w_1$

Σ is symmetric

Principal Components Analysis (PCA)



- Second principal component: $\max \text{Var}(z_2)$, s.t. $\|w_2\| = 1$ and w_2 is orthogonal to w_1

$$\max_{w_2} w_2^T \Sigma w_2 - \alpha (w_2^T w_2 - 1) - \beta (w_2^T w_1 - 0)$$

- The derivative **with respect to** w_2 should be 0

$$(*) \quad 2\Sigma w_2 - 2\alpha w_2 - \beta w_1 = 0$$

- Multiply from left by w_1^T

$$2w_1^T \Sigma w_2 - 2\alpha w_1^T w_2 - \beta w_1^T w_1 = 0$$

$$2w_1^T \Sigma w_2 - \beta = 0$$

- $w_1^T w_2 = 0$, $w_1^T w_1 = 1$, $w_1^T \Sigma w_2$ is a scalar and equals to its transpose

$$w_1^T \Sigma w_2 = w_2^T \Sigma w_1 = w_2^T \lambda_1 w_1 = \lambda_1 w_2^T w_1 = 0$$

- Then $\beta = 0$ and from $(*)$ we have $\Sigma w_2 = \alpha w_2$ and w_2 should be an eigenvector of Σ
- Similarly, for $w_3, w_4, \dots$

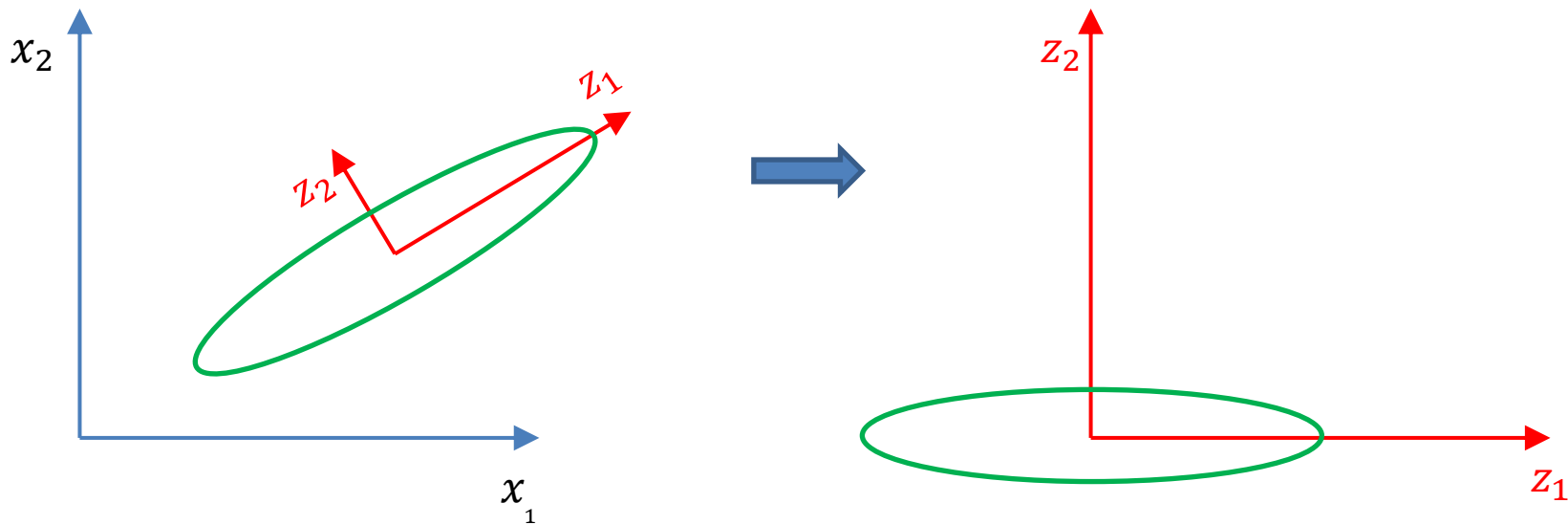
What PCA Does?



$$z = \mathbf{W}^T (x - \mu)$$

where the columns of $\mathbf{W}$ are the eigenvectors of Σ , and μ is the sample mean

It centers the data at the origin and rotates the axes



How to Choose the Number of Principal Components?



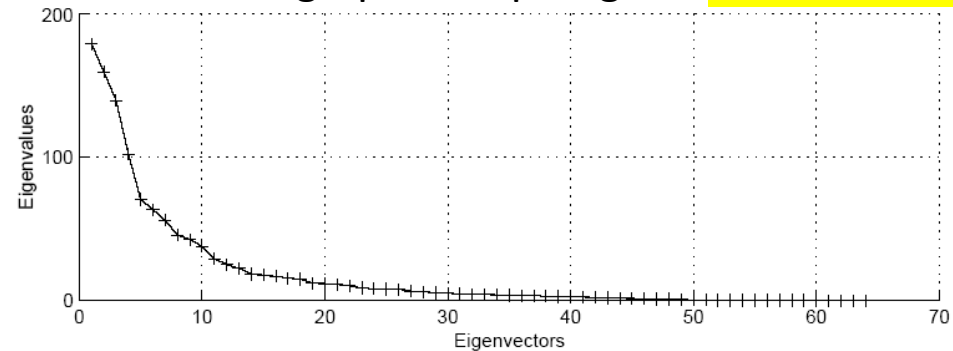
- In dimensionality reduction we use only k , $k < d$ principal components
- **Proportion of Variance** (PoV) explained, when eigenvalues λ_i are sorted in descending order

$$\frac{\lambda_1 + \lambda_2 + \dots + \lambda_k}{\lambda_1 + \lambda_2 + \dots + \lambda_k + \dots + \lambda_d}$$

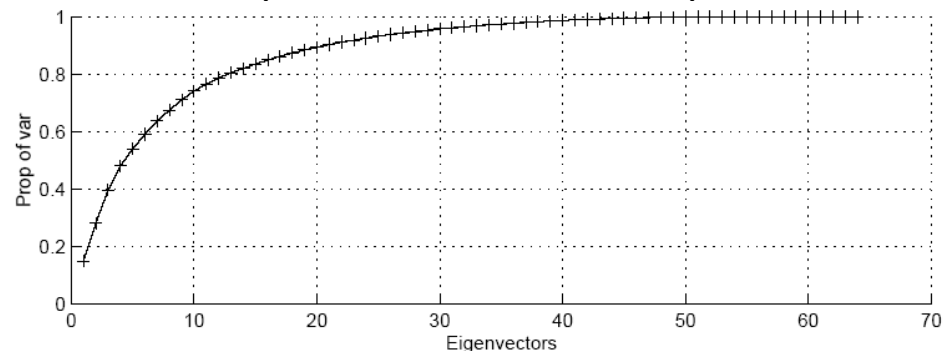
- Typically, stop at $PoV > 0.9$
- **Scree** graph plots of *eigenvalues* vs k , stop at “elbow”

Scree graph for optdigits

SK: Sutinový graf



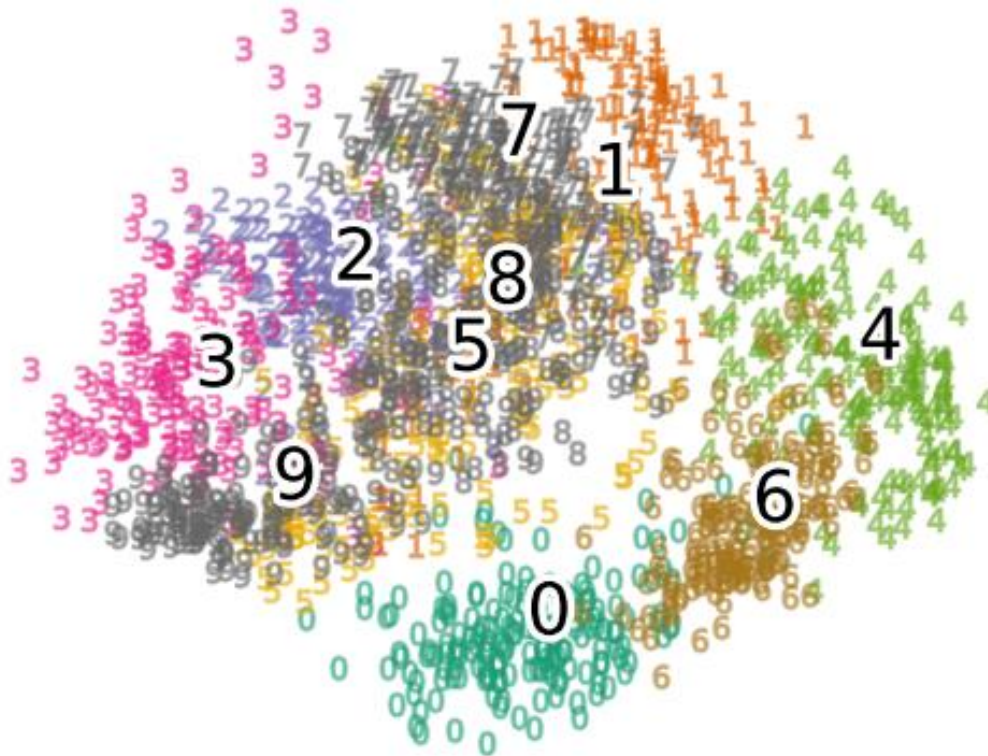
Proportion of variance explained



PCA example



Principal component analysis (time 0.006s)



Pictures of digits
(8 × 8 pixels)



Factor Analysis

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_d) = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{Nd} \end{pmatrix}$$

- Find a small number of **factors** $\mathbf{z}_1, \dots, \mathbf{z}_k \in \mathbb{R}^N$, which when combined generate $\mathbf{x}$:

$$\mu_i = \frac{1}{N} \sum_{j=1}^N x_{ji}$$

$$\mathbf{x}_i - \mu_i = v_{i1}\mathbf{z}_1 + v_{i2}\mathbf{z}_2 + \dots + v_{ik}\mathbf{z}_k + \boldsymbol{\varepsilon}_i$$

where $\mathbf{z}_j, j = 1, \dots, k$ ($k < d$) are the **latent factors** with

$$E[\mathbf{z}_j] = 0, \text{Var}(\mathbf{z}_j) = 1, \text{Cov}(\mathbf{z}_i, \mathbf{z}_j) = 0, i \neq j,$$

$\boldsymbol{\varepsilon}_i \in \mathbb{R}^N$ are the **noise sources**

$$E[\boldsymbol{\varepsilon}_i] = \boldsymbol{\psi}_i; \text{Cov}(\boldsymbol{\varepsilon}_i, \boldsymbol{\varepsilon}_j) = 0, i \neq j; \text{Cov}(\boldsymbol{\varepsilon}_i, \mathbf{z}_j) = 0, \forall i, j$$

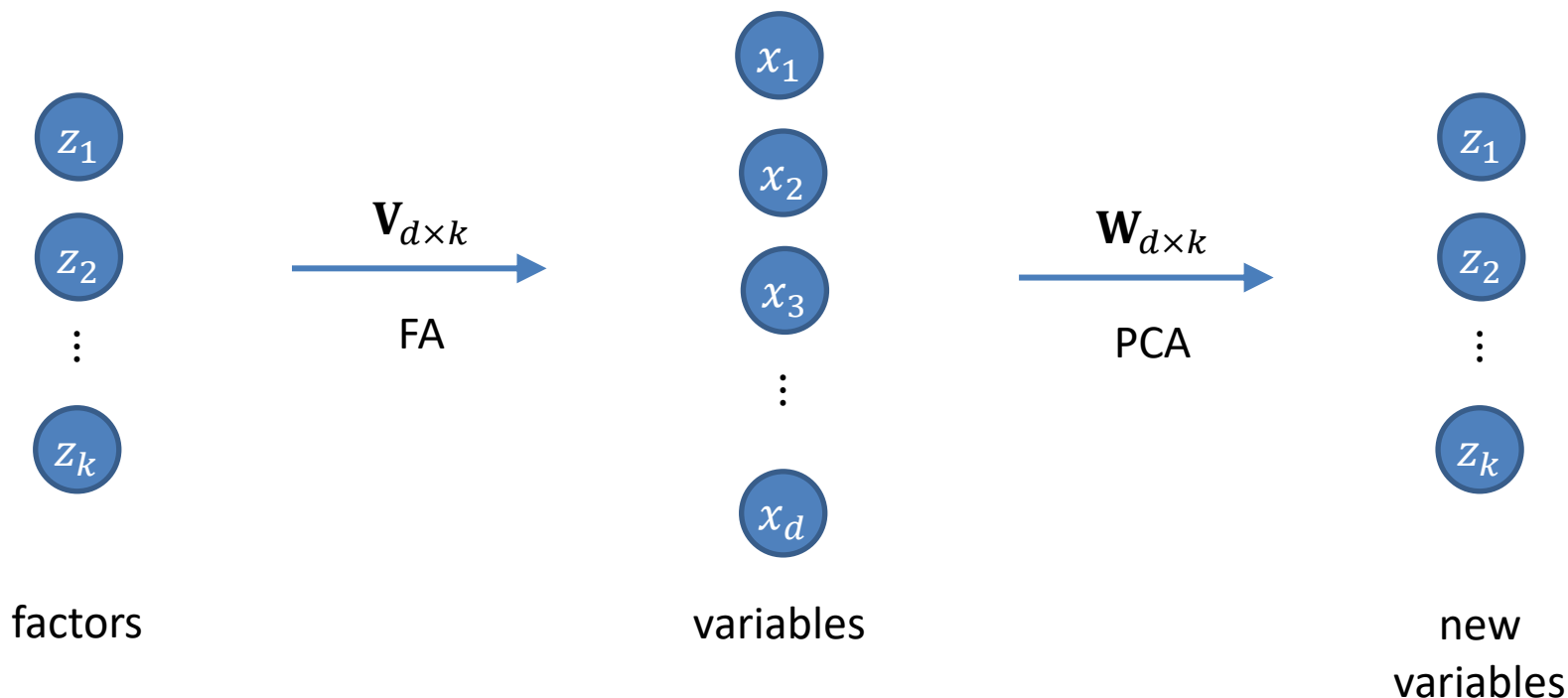
and v_{ij} are the **factor loadings**, $\mathbf{V} = (v_{ij})_{d \times k}$

PCA vs FA



How to transform a **vector** x into new vector z ?

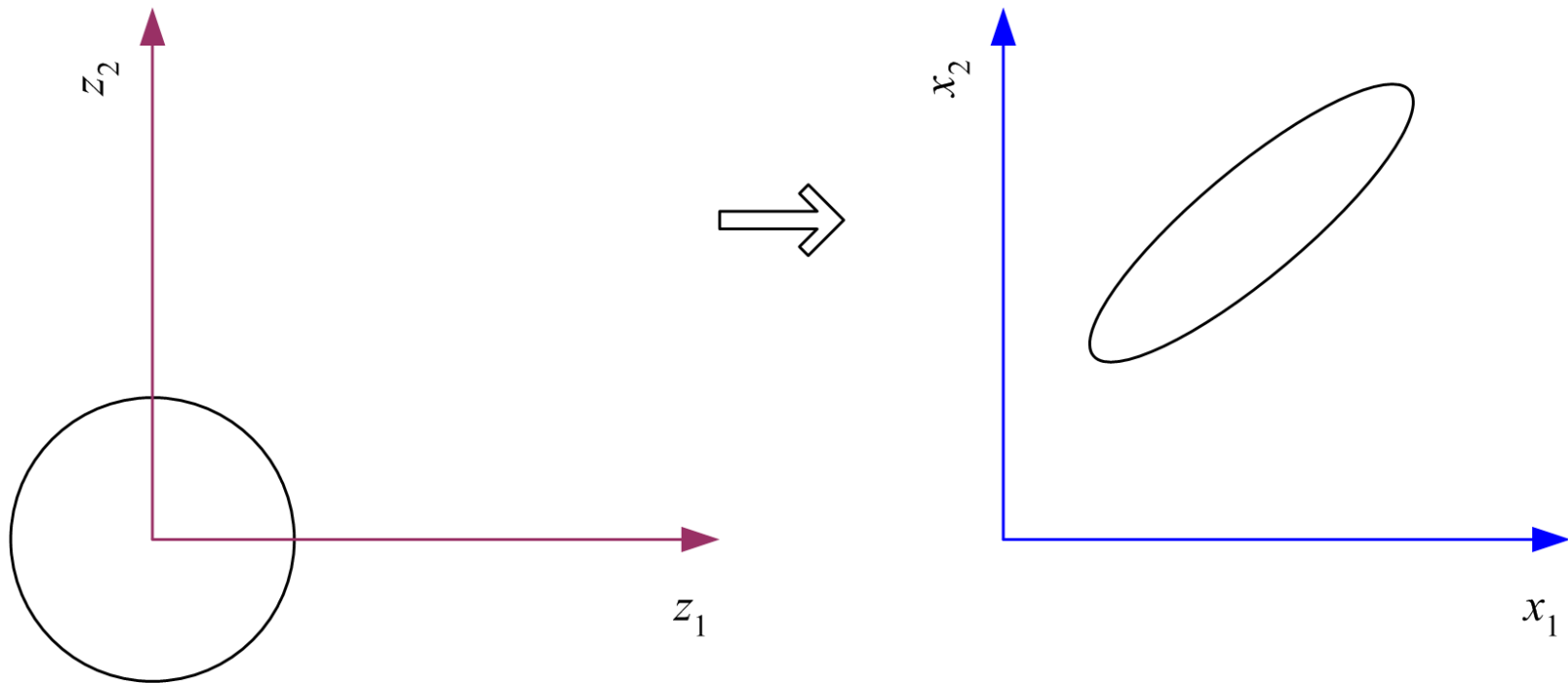
- FA From z to x $x - \mu = \mathbf{V}z + \varepsilon$
- PCA From x to z $z = \mathbf{W}^T(x - \mu)$



Factor Analysis



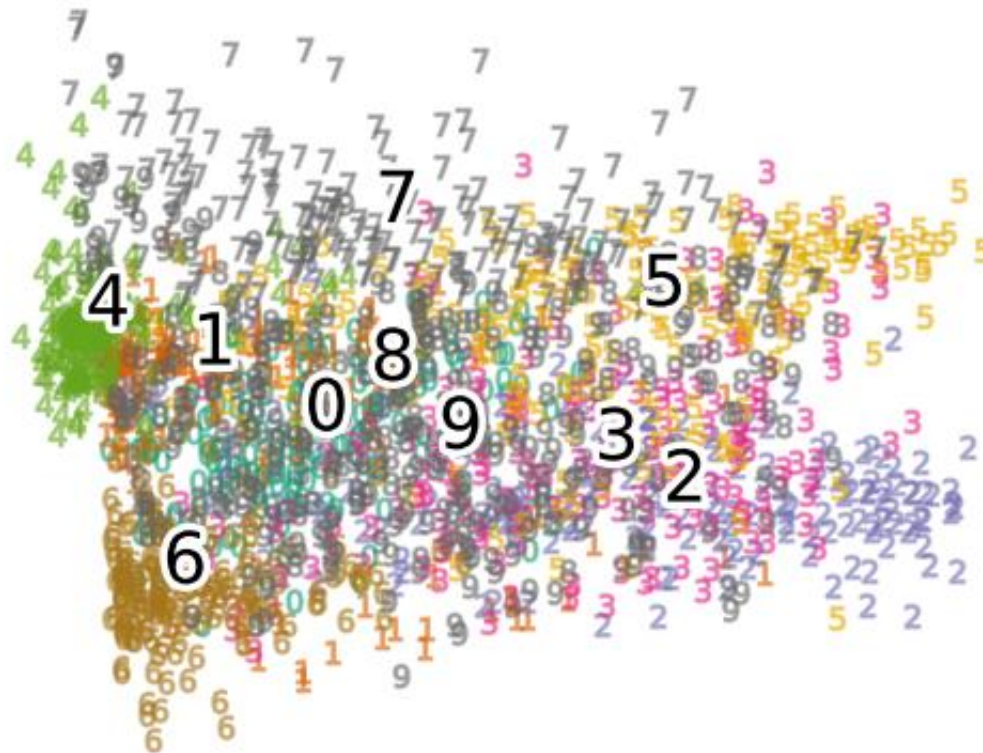
- In FA, factors z_j are stretched, rotated and translated to generate x



Factor Analysis Example



Factor analysis (time 0.105s)



Multidimensional Scaling



- Given pairwise distances between N points

$$d_{i,j}^* = \|x_i - x_j\| \quad i, j = 1, \dots, N$$

place the points on a low-dim map s.t. distances are preserved.

- $z = g(x \mid \theta)$
 - Classical MDS: $z = g(x \mid \theta) = \mathbf{W}^T x$, i.e., $\theta = \mathbf{W}$ (in `scikit-learn MDS(metric=True)`)
 - In general: find θ that minimizes **Sammon stress**

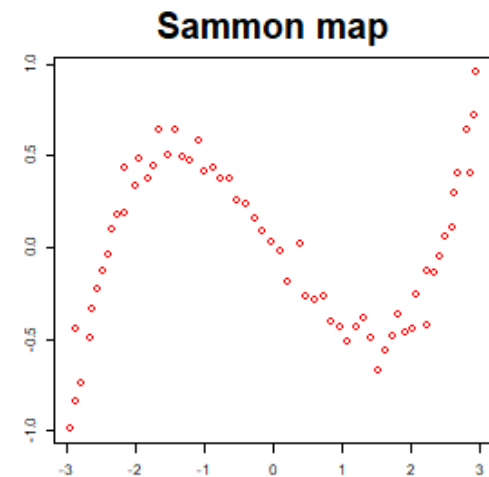
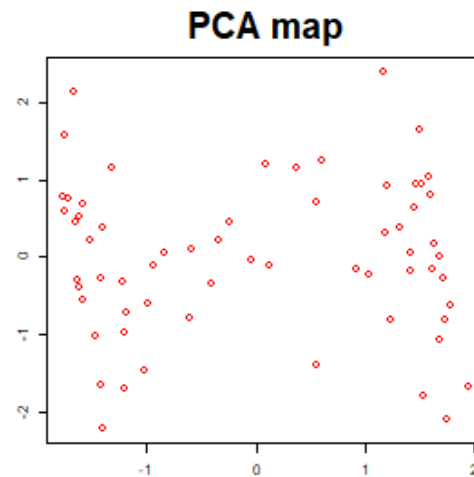
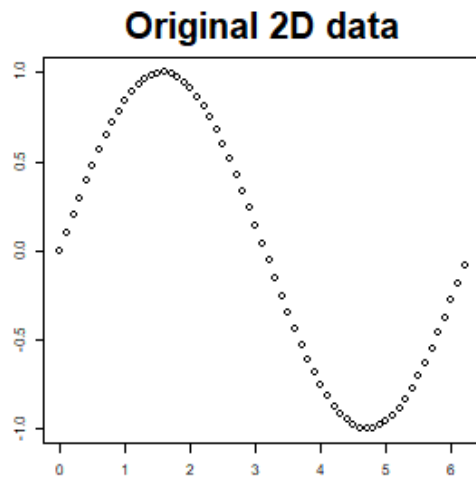
$$E[\theta \mid X] = \frac{1}{\sum_{i < j} d_{i,j}^*} \sum_{i=1}^N \sum_{j=i+1}^N \frac{(d_{i,j}^* - \|z_i - z_j\|)^2}{d_{i,j}^*}$$

$E[\theta \mid X]$ is minimized using gradient descent methods or some other iterative method – **a nonlinear mapping!** (in `scikit-learn MDS(metric=False)`)

Multidimensional Scaling



- Example from Z. Yang: *Machine Learning Approaches to Bioinformatics*
 - The points in 2D were randomly “shifted” from a plane in 3rd dimension



Linear Discriminant Analysis

- a **supervised** method for dimensionality reduction **for classification problems**
- Find a low-dimensional (1-dimensional) space such that when x is projected to $w^T x$, the classes are well-separated.
- Find w that **maximizes**

Fisher's linear discriminant

$$J(w) = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2}$$

labels

- We are given a sample $X = \{(x_t, r_t)\}_{t=1}^N$ such that

$$r_t = \begin{cases} 1 & \text{if } x_t \in C_1 \\ 0 & \text{if } x_t \in C_2 \end{cases}$$

$$m_1 = w^T \bar{m}_1 = \frac{\sum_t w^T x_t r_t}{\sum_t r_t}$$

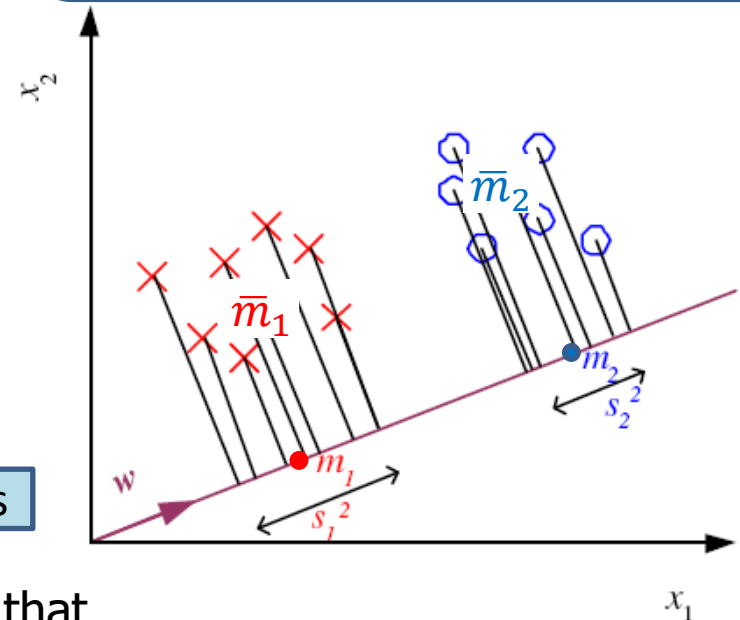
$$m_2 = w^T \bar{m}_2 = \frac{\sum_t w^T x_t (1 - r_t)}{\sum_t (1 - r_t)}$$

$$s_1^2 = \sum_t (w^T x_t - m_1)^2 r_t$$

$$s_2^2 = \sum_t (w^T x_t - m_2)^2 (1 - r_t)$$

Variances after projection

$\bar{m}_1$ and $\bar{m}_2$ are the centers of classes
 s_1^2, s_2^2 correspond (are not equal) to the projected variances of the classes **after mapping**



Linear Discriminant Analysis



- Between-class scatter:

$$\begin{aligned}(m_1 - m_2)^2 &= (w^T \bar{m}_1 - w^T \bar{m}_2)^2 \\ &= w^T (\bar{m}_1 - \bar{m}_2) (\bar{m}_1 - \bar{m}_2)^T w \\ &= w^T \mathbf{S}_B w \text{ where } \mathbf{S}_B = (\bar{m}_1 - \bar{m}_2) (\bar{m}_1 - \bar{m}_2)^T\end{aligned}$$

maximized

- Within-class scatter:

$$\begin{aligned}s_1^2 &= \sum_t (w^T x_t - m_1)^2 r_t \\ &= \sum_t w^T (x_t - \bar{m}_1) (x_t - \bar{m}_1)^T w r_t = w^T \mathbf{S}_1 w\end{aligned}$$

minimized

where $\mathbf{S}_1 = \sum_t (x_t - \bar{m}_1) (x_t - \bar{m}_1)^T r_t$ for $\mathbf{S}_2$ analogically

$$s_1^2 + s_2^2 = w^T \mathbf{S}_W w \text{ where } \mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2$$

Fisher's Linear Discriminant



- Find w that maximizes

$$J(w) = \frac{w^T \mathbf{S}_B w}{w^T \mathbf{S}_W w} = \frac{|w^T (\bar{m}_1 - \bar{m}_2)|^2}{w^T \mathbf{S}_W w}$$

Maximize between-class scatter

Minimize within-class scatter

- LDA solution (the derivative of $J(w)$ must be zero):

$$w = c \cdot \mathbf{S}_W^{-1} (\bar{m}_1 - \bar{m}_2) \text{ for some constant } c$$

we can take $c = 1$

Normal distribution with mean μ_i and covariance matrix Σ

- Parametric solution: $w = \Sigma^{-1}(\mu_1 - \mu_2)$ when $p(x|C_i) \sim \mathcal{N}(\mu_i, \Sigma)$
- Moreover, this solution can be used also when the classes are not normal

K>2 Classes



- Classes $C_1, \dots, C_K$
$$r_t^{(i)} = \begin{cases} 1 & \text{if } x_t \in C_i \\ 0 & \text{if } x_t \notin C_i \end{cases}$$

$$\mathbf{X} = \left\{ \left(x_t, r_t^{(1)}, \dots, r_t^{(K)} \right) \right\}_{t=1}^N$$

- We want to map d -dimensional space into k -dimensional space

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} \quad \mathbf{W} \in \mathbb{R}^{d \times k}, \mathbf{z} \in \mathbb{R}^k$$

- Total within-class scatter matrix:

$$\mathbf{S}_W = \sum_{i=1}^K \mathbf{S}_i \quad \mathbf{S}_i = \sum_t r_t^{(i)} (x_t - m_i)(x_t - m_i)^T$$

Within-class scatter
matrix for C_i

- Between-class scatter matrix:

$$\mathbf{S}_B = \sum_{i=1}^K N_i (m_i - m)(m_i - m)^T \quad m = \frac{1}{K} \sum_{i=1}^K m_i \quad N_i = \sum_{t=1}^N r_t^{(i)}$$

How the class means are scattered
around the overall mean

K>2 Classes



- The between-class and within-class scatter matrices after projection are

$$\mathbf{W}^T \mathbf{S}_B \mathbf{W} \quad \text{and} \quad \mathbf{W}^T \mathbf{S}_W \mathbf{W}$$

- Find $\mathbf{W}$ that maximizes

$$J(\mathbf{W}) = \frac{|\mathbf{W}^T \mathbf{S}_B \mathbf{W}|}{|\mathbf{W}^T \mathbf{S}_W \mathbf{W}|}$$

- The solution are the largest eigenvectors of $\mathbf{S}_W^{-1} \mathbf{S}_B$

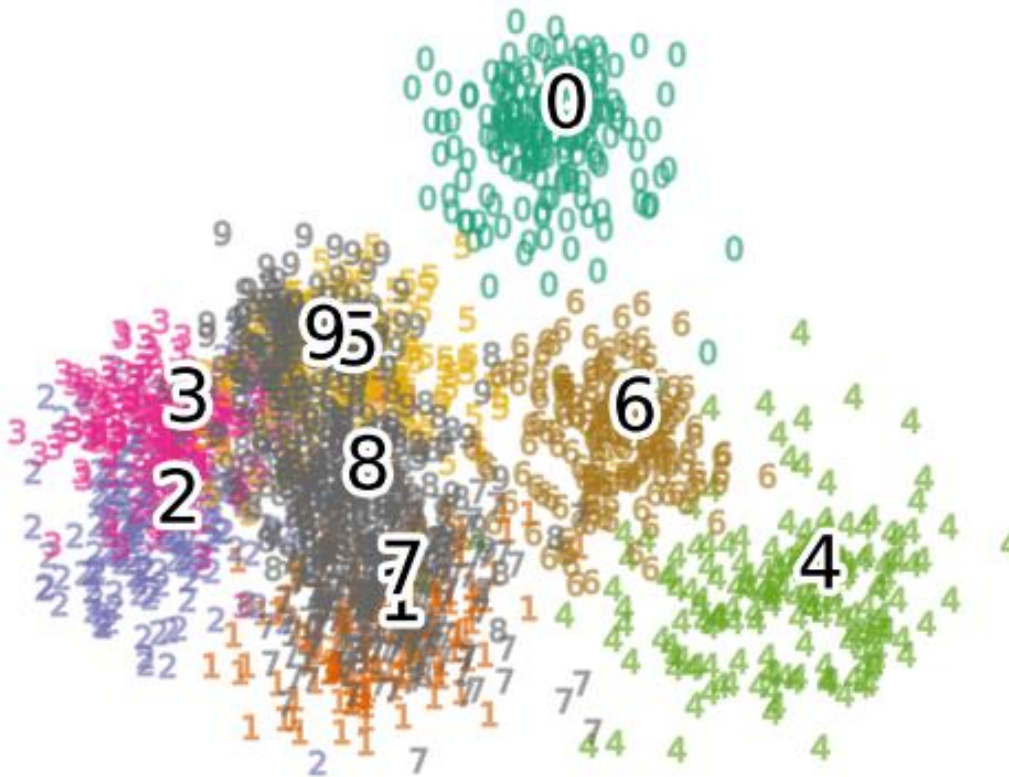
$$\mathbf{S}_B = \sum_{i=1}^K N_i (m_i - m)(m_i - m)^T$$

is a sum of K matrices of rank 1, only $K - 1$ of them are independent, therefore we take $k = K - 1$

Linear Discriminant Analysis



Linear Discriminant Analysis embedding (time 0.024s)



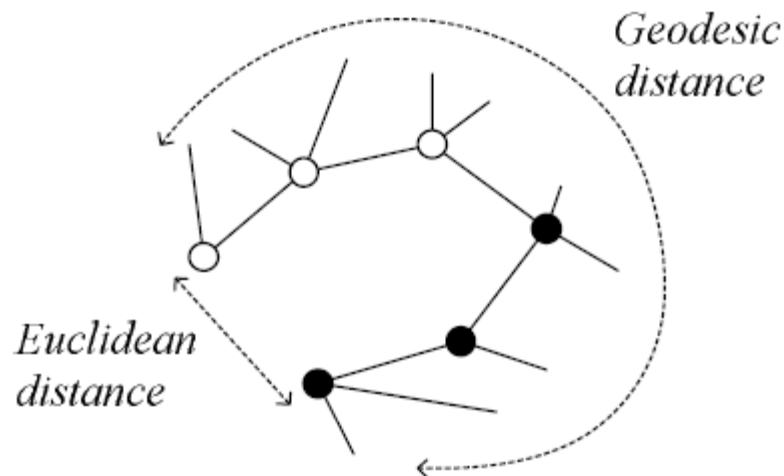
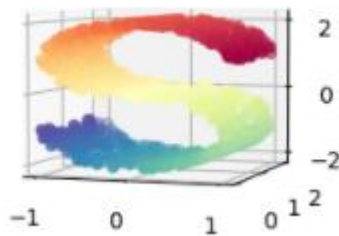
Pictures of digits

Note that the classes are better separated than in the case of PCA

Isomap



- Geodesic distance is the distance along the manifold that the data lies in, as opposed to the Euclidean distance in the input space



Isomap

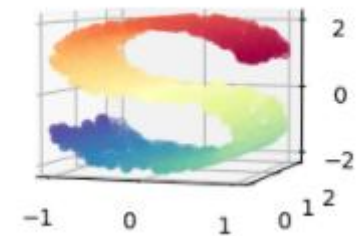
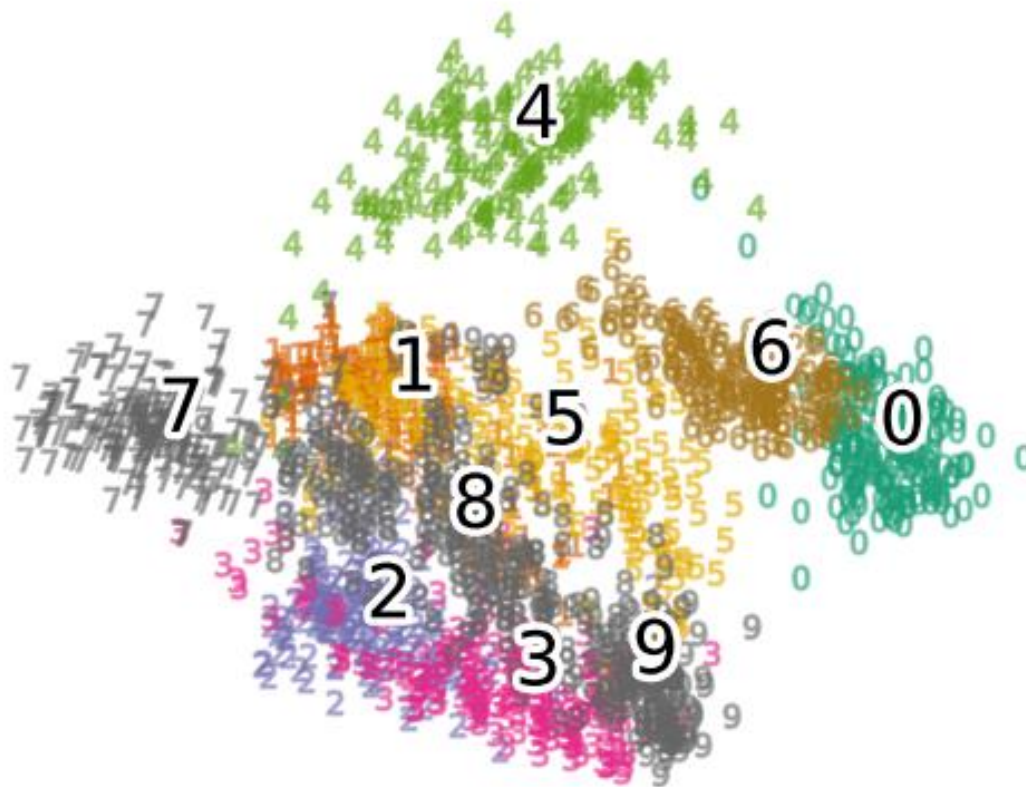


- Instances r and s are connected in the graph if $\|x_r - x_s\| < \varepsilon$ or if x_s is one of the k neighbors of x_r . The edge length is $\|x_r - x_s\|$
- For two nodes r and s not connected, the distance is equal to the shortest path between them
- Once the $N \times N$ distance matrix is thus formed, use MDS to find a lower-dimensional mapping

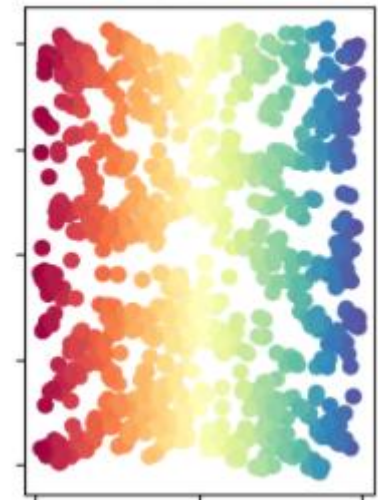
Isomap – Example



Isomap embedding (time 2.790s)



Isomap (0.56 sec)



Locally Linear Embedding



1. Given x_r find its S closest neighbors $x_s^{(r)}, s = 1, \dots, S$
2. Find $\mathbf{W} = (W_{rs})_{r=1, \dots, N, s=1, \dots, S}$ that minimize error in the original space

$$E(\mathbf{W} | X) = \sum_r \left\| x_r - \sum_s W_{rs} x_s^{(r)} \right\|^2$$

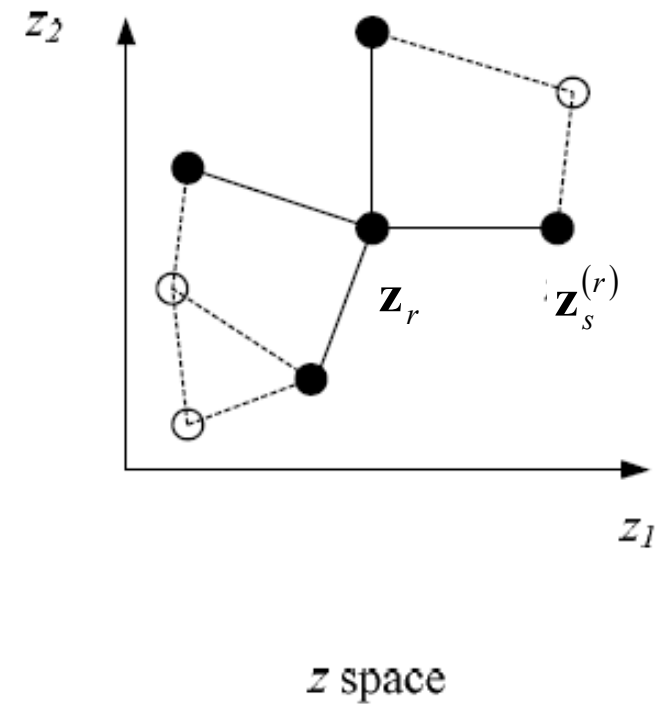
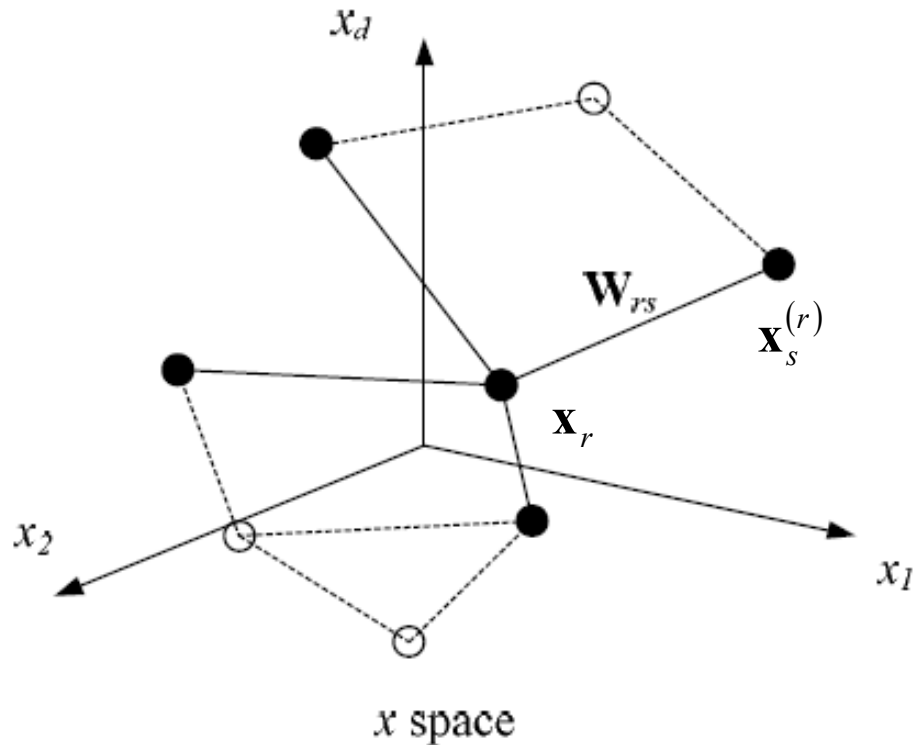
using least squares, subject to

$$W_{rr} = 0, \quad \sum_s W_{rs} = 1, \quad \forall r$$

3. Find the new coordinates z_r in a lower dimensional space that minimize

$$E(z | \mathbf{W}) = \sum_r \left\| z_r - \sum_s W_{rs} z_s^{(r)} \right\|^2$$

Locally Linear Embedding

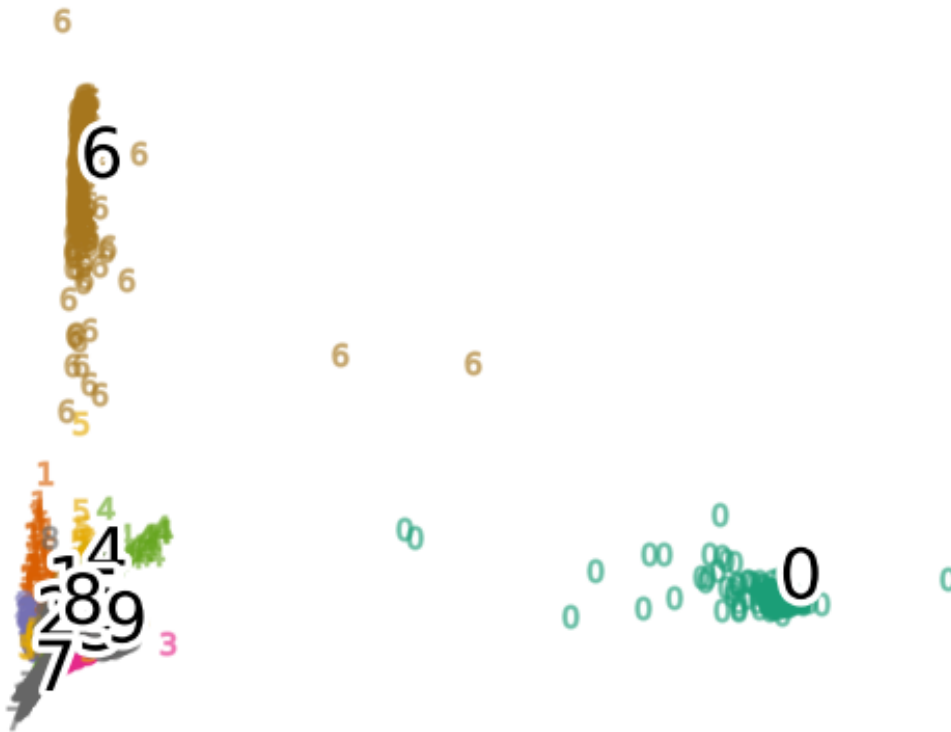


Locally Linear Embedding on Optdigits



LTSA LLE embedding (time 4.620s)

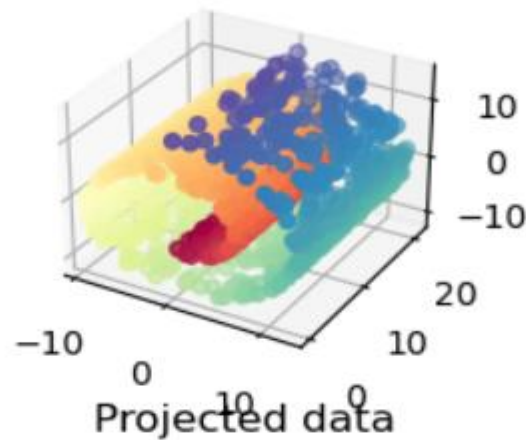
Pictures of digits



Locally Linear Embedding on Swiss roll



Original data



t -SNE – t -distributed Stochastic Neighbor Embedding



- [Laurens van der Maaten and Geoffrey Hinton, 2008]
 - Nonlinear transformation from a high-dimensional space into a lower-dimensional space with different transformations on different regions
 - A perplexity parameter $\approx$ “a guess about the number of close neighbors each point has”
1. It constructs a probability distribution over pairs of high-dimensional objects in such a way that similar objects are assigned a higher probability while dissimilar points are assigned a lower probability.
 2. t -SNE defines a similar probability distribution over the points in the low-dimensional map, and it minimizes the Kullback–Leibler divergence (KL divergence) between the two distributions with respect to the locations of the points in the map.
- Usually used for mapping into 2 or 3 dimensions

t -SNE – t -distributed Stochastic Neighbor Embedding



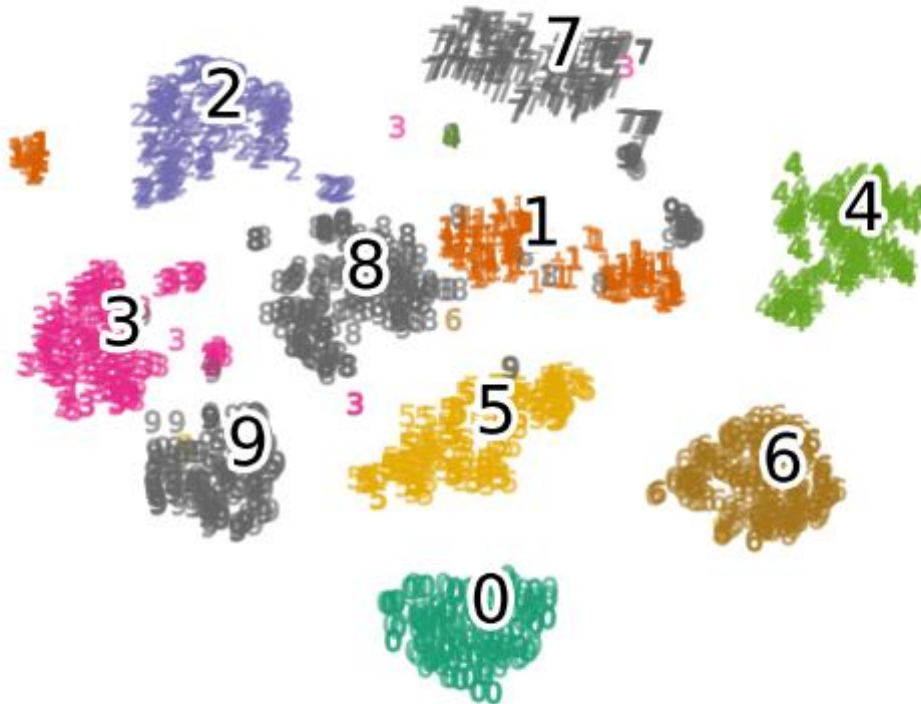
- Parameters:
 1. Perplexity
 2. Early exaggeration factor
 3. Learning rate
 4. Maximum number of iterations
- Cons:
 - computationally expensive
 - stochastic and multiple restarts with different seeds can yield different embeddings
 - Does not preserve the global structure
- See [“How to Use t-SNE Effectively”](#) for a discussion of various parameter settings

t -SNE – t -distributed Stochastic Neighbor Embedding



Pictures of digits

t-SNE embedding (time 3.042s)



UMAP – Uniform Manifold Approximation and Projection



- [‘18] Leland McInnes, John Healy, James Melville: **UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction**, online <https://arxiv.org/abs/1802.03426>
- See
 - Understanding UMAP <https://pair-code.github.io/understanding-umap/>
 - UMAP documentation https://umap-learn.readthedocs.io/en/latest/how_umap_works.html
- Two steps:
 1. construct a weighted graph in high dimensions
 - Edges to k nearest neighbors and
 - an edge weight = how close are the vectors
 - <https://pair-code.github.io/understanding-umap/supplement.html#:~:text=some%20theoretical%20guarantees,with%20less%20weight%3B>
 2. optimize to find the most similar graph in lower dimensions.

UMAP – Uniform Manifold Approximation and Projection



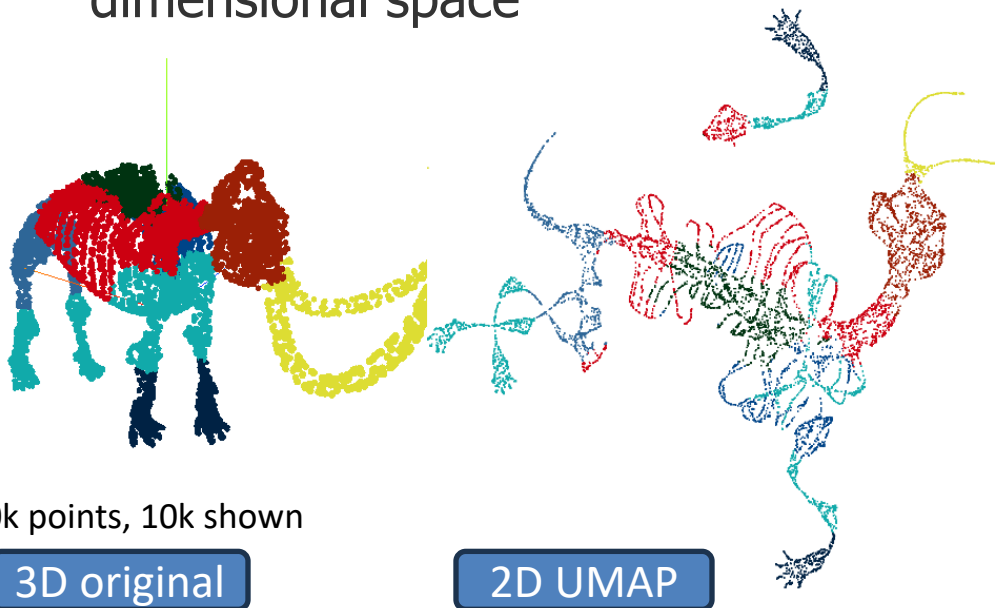
1. Construct a weighted graph in high dimensions
 - extend a radius outwards from each point, connecting points when those radii overlap; small radius $\Rightarrow$ isolated clusters, large radius $\Rightarrow$ complete graph
 - Choose the radius locally = the distance to k^{th} nearest neighbor
 - Make the graph "fuzzy" – decrease the likelihood of connection as the radius grows.
 - ensure that each point is connected to at least its closest neighbor
2. Optimize to find the most similar graph in lower dimensions
 - Employ a force-directed graph layout algorithm

UMAP – Uniform Manifold Approximation and Projection



The most useful parameters:

- `n_neighbors=k` the number of nearest neighbors used to construct the initial high-dimensional graph
- `min_dist`, or the minimum distance between points in low-dimensional space

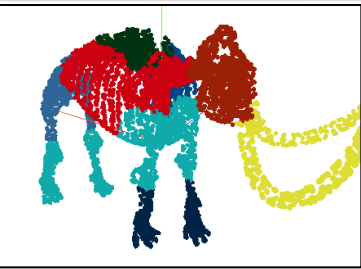


`n_neighbors=200`
`min_dist=0.25`

50k points, 10k shown

Andy Coenen, Adam Pearce > <https://pair-code.github.io/understanding-umap/index.html>

UMAP – Uniform Manifold Approximation and Projection

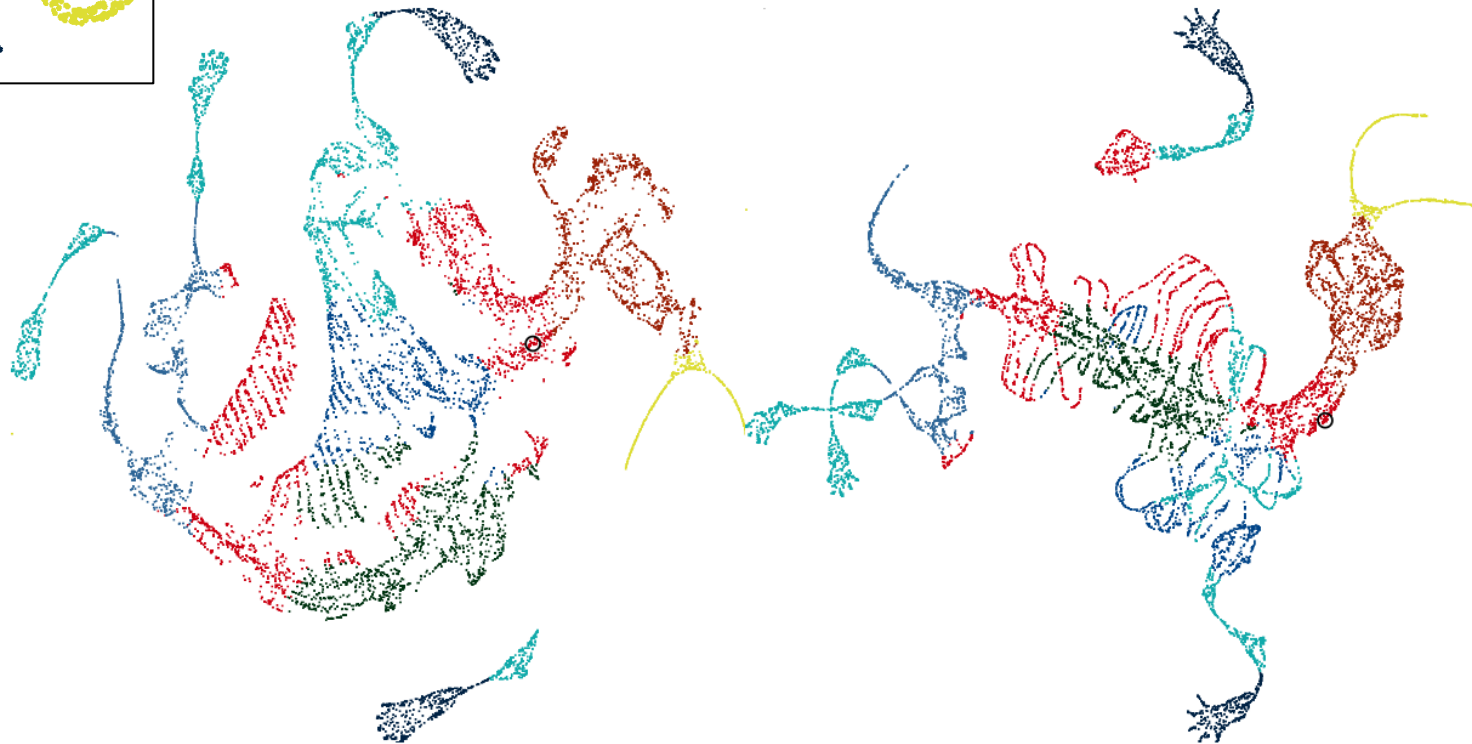


2D t-SNE

time: 42m 24s

2D UMAP

time: 3m 22s



Andy Coenen, Adam Pearce > <https://pair-code.github.io/understanding-umap/index.html>

UMAP – Uniform Manifold Approximation and Projection



UMAP (time 5.097s)



Pictures of digits